TRUSTED MEMORY PIPELINE

AI Project Memory

V0.2.1 + V0.3 联合开发报告

Q/A 单元 · 哈希缓存 · 标签复核 · 总结框 · 时间线 · Context Package

01	可信底座	原始消息不被摘要覆盖
02	逐对话分析	Q/A 与缓存控制成本
03	人在回路	相似标签不静默合并
04	本地体验	总结、证据、时间线一体

版本 0.2.1 / 0.3.0 | 2026-09-13 | 本地产品 Alpha

目录

1. 交付结论 3
2. 为什么现在不需要训练模型 3
3. 正确的数据流水线 3
4. V0.2.1：Q/A 单元怎样产生 4
5. 哈希到底负责什么 4
6. 标签归并的安全策略 4
7. V0.2.1 的数据库变化 5
8. V0.3：总结框与本地界面 5
9. 时间线与 Context Package 5
10. V0.3 新增接口 6
11. 实际运行方法 6
12. 本轮代码与文件结构 6
13. 自动测试与手动验收 7
14. 隐私、权限与发布边界 7
15. 你现在需要人工做什么 7
16. 版本成熟度判断 7
17. 下一步路线与完整发布预估 8
18. 本轮最终边界 8
19. 下一轮人工验收记录模板 9

1. 交付结论

本轮已经连续完成两个版本。V0.2.1 建立了“按对话拆 Q/A、保留证据、利用哈希缓存、保守归并标签”的可信知识底座；V0.3 在这个底座上增加了本地三栏总结界面、Q/A 视图、时间线、人工审核和 Context Package。

你的判断是正确的：如果只增加 Q/A 表和缓存，它仍是 V0.2.1；当系统开始让普通用户在浏览器里选择主题、阅读总结、核对原句、批准内容并导出上下文，它就已经进入 V0.3。本轮没有把两个阶段混成一个虚假的大版本，而是先完成底层再接界面。

验收项	实际结果	状态
自动测试	15 / 15 通过	通过
示例项目	1 个项目、2 条对话	通过
Q/A 单元	2 个	通过
Q/A-句子证据	8 条	通过
Q/A-主题关系	6 条	通过
知识主题	5 个	通过
主题-句子证据	16 条	通过
“学习方法”跨对话证据	2 条对话、6 个句子	通过
本地 Web 页面	Dashboard、三栏总结、Q/A、时间线	通过
审核写入	approve / reject / reset	通过
上下文包	摘要、Q/A、原句、来源坐标	通过

2. 为什么现在不需要训练模型

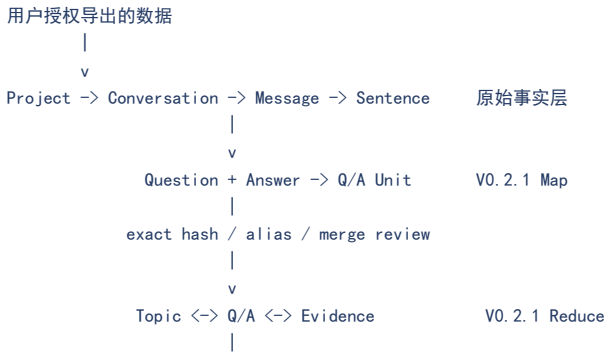
这个产品当前需要的是一个“可靠的处理系统”，而不是自己训练一个大模型。导入、拆分、哈希、缓存、证据绑定、主题文件、审核和版本历史都属于普通软件与数据库能力；理解问题、生成摘要和提出主题候选，可以调用现有 LLM 完成。

当前推荐组合是：现有 LLM + 结构化输出 + 本地数据库 + 精确哈希 + 相似度候选 + 人工复核。只有在积累了稳定分类体系、数千条人工标注样本、明确失败模式和评测集以后，微调或训练小模型才有经济意义。

本轮默认流程仍完全离线。离线规则的职责是验证产品闭环，不冒充成熟语义模型。用户未来显式选择 OpenAI 提取器、模型 ID 和自己的 API Key 时，才会发生外部模型请求。

3. 正确的数据流水线

系统采用 Map-Reduce 思路。Map 阶段逐条对话处理，避免一次把所有聊天塞进模型；Reduce 阶段再把跨对话 Q/A 与句子归入主题，生成知识文件。



```
                          v
Knowledge File -> Revision -> Human Review          V0.2
                          |
                          v
Summary UI / Timeline / Context Package             V0.3
```

最重要的约束是：派生内容永远不能覆盖原始消息。即使 AI 摘要错误，用户仍能沿 topic -> Q/A -> sentence -> message -> conversation -> project 回到来源。

4. V0.2.1：Q/A 单元怎样产生

系统先读取一条对话中的问题及直接回答。每一对问题与回答形成稳定 Q/A ID，保存问题原文、回答原文、候选摘要、内容 SHA-256、处理器版本和状态。问题句与回答句分别写入证据关系。

一个大问题可以继续拆成多个 Q/A 单元；一个 Q/A 也可以属于多个主题。例如示例中的“如何安排每周复习计划？我希望兼顾专业课和英语”同时关联学习规划、学习方法、专业课学习和英语学习，但它的原始位置仍只有一个。

对象	权威程度	能否被 AI 改写	作用
Message / Sentence	原始事实	不允许	保存原话与位置
Question / Answer link	结构事实	只允许重新识别	表示问答关系
Q/A Unit summary	派生候选	允许重算	便于浏览和归纳
Topic label	派生候选	允许归并	建立跨对话结构
Knowledge document	阅读投影	允许修订	供人学习与复用
Review event	人工判断	不覆盖历史	记录批准或退回

5. 哈希到底负责什么

哈希不是语义理解工具。V0.2.1 使用 SHA-256 处理精确身份、重复数据、缓存与修订：文件哈希判断同一导出是否已导入；句子哈希标记正文；Q/A 内容哈希标记问题与回答组合；规范化标签哈希标记精确别名；对话来源哈希与处理配置共同形成缓存键；知识内容哈希决定是否创建新修订。

```
cache_key = SHA256(
  conversation_id
  + conversation_source_hash
  + extractor_version
  + prompt_version
  + model_name
)
```

“学习 方法”和“学习方法”规范化后可得到相同哈希；“学习技巧”和“学习方法”即使人认为接近，也会得到不同哈希。后者必须进入语义候选或人工判断，不能靠哈希放进同一桶。

6. 标签归并的安全策略

标签解析分成三层。第一层是规范化后的精确命中，自动使用已有主题。第二层是已经批准的别名，例如人工确认“复习规划”属于“学习规划”，以后可以直接复用。第三层是相似候选，只生成 merge review。

判断结果	系统动作	是否需要人
精确规范化标签相同	直接关联已有主题	否

判断结果	系统动作	是否需要人
命中已确认别名	直接关联别名指向主题	否
文字相似但未确认	建立待判断任务	是
明显不同	建立新候选主题	后续审核

人工选择 merge 后，Q/A 关系与别名迁移到目标主题，候选主题标记为 merged；选择 keep_separate 后，两者继续独立。相似度绝不会单独成为最终事实。

7. V0.2.1 的数据库变化

本轮数据库版本升到 4，保留旧数据并自动补充编译配置字段。新增六张表：analysis_cache、qa_units、qa_evidence、topic_aliases、qa_topics、merge_reviews。

表	关键字段	解决的问题
analysis_cache	cache_key、source_sha256、prompt_version	相同对话不重复处理或付费
qa_units	question_text、answer_text、content_sha256	把混杂对话拆成可复用问答
qa_evidence	qa_unit_id、sentence_id、relation	回到问题句和回答句
topic_aliases	normalized_alias、alias_sha256、topic_id	保存精确别名与人工决定
qa_topics	qa_unit_id、topic_id、confidence	支持一个问答属于多个主题
merge_reviews	candidate、suggested、similarity、status	阻止静默错误合并

KnowledgeCompiler 还增加了 config_fingerprint。现在不仅输入原句发生变化会触发重新编译，提取器规则或输入限制等配置变化也能被识别。旧数据库在 initialize 时会安全补齐字段。

8. V0.3：总结框与本地界面

V0.3 的页面不是装饰，它是人工审核闭环。页面顶端展示项目、对话、Q/A、主题和待审核数量；左侧是主题目录；中间是总结框；右侧是逐句证据；底部是 Q/A 单元和消息时间线。

总结框严格分开“核心归纳”与“AI 扩展（非原始证据）”。用户点击主题后，可以看到它跨越了多少条对话和多少个证据句。右侧每条证据显示项目、对话、角色和 sentence_id。

页面使用纯 HTML、CSS 和 JavaScript，不依赖前端构建工具。这样初学者可以直接运行，也减少 V0.3 的安装复杂度。视觉上采用低干扰的纸张色和深绿色，把注意力放在知识与证据，而不是聊天气泡。

9. 时间线与 Context Package

时间线按发生时间展示消息和审核事件，并保留 project_id、conversation_id 与标题。当前它先回答“当时发生了什么”；后续版本再增加“观点发生了什么变化”和冲突标记。

Context Package 是跨对话与跨 AI 使用的关键产品对象。它不是偷偷赋予某个平台账号权限，而是由本地系统把用户已经授权导入的主题摘要、相关 Q/A、原句证据和 AI 扩展组装成 Markdown。用户可以检查后交给另一个 AI，形成可控、透明、可撤回的上下文传递。

Context Package 当前包含：主题标题与摘要；关联 Q/A；每个 Q/A 的来源项目与对话；逐句证据和 sentence_id；单独标记的 AI 扩展。示例文件已生成在 output/context/学习方法.md。

10. V0.3 新增接口

请求	作用	数据变化
GET /v0/dashboard	获取产品总览	无
GET /v0/qa-units	查看 Q/A 单元	无
GET /v0/merge-reviews	查看归并候选	无
POST /v0/merge-reviews/{id}	合并或保持分开	有，人工决定
GET /v0/timeline	查看项目时间线	无
GET /v0/context/{topic}	生成上下文包	无
POST /v0/refresh	重新分析和离线编译	有，生成派生数据
POST /v0/review	批准或退回总结	有，记录审核事件

V0.1 与 V0.2 的项目、问题、搜索、消息、主题、知识文件和修订接口继续保留。接口草案位于 contracts/openapi-v0.3.yaml。

11. 实际运行方法

第一次体验可以使用项目内的合成数据，不涉及个人聊天。

```
python -m ai_memory init
python -m ai_memory import examples/learning-project.json
python -m ai_memory compile
python -m ai_memory serve
```

浏览器打开 http://127.0.0.1:8765。点击“学习方法”可以看到它从“学习方法”和“学习规划”两条对话吸收 6 条证据。点击“确认可用”记录人工批准；点击“生成上下文包”显示可交给另一个 AI 的 Markdown。

单独检查底层时可使用：

```
python -m ai_memory analyze
python -m ai_memory qa
python -m ai_memory merge-reviews
python -m ai_memory timeline
python -m ai_memory context 学习方法 --output output/context/学习方法.md
```

12. 本轮代码与文件结构

```
GPT-SaaS/
├── ai_memory/
│   ├── qa.py                 # V0.2.1 Q/A、缓存、标签与归并
│   ├── experience.py         # V0.3 Dashboard、时间线、上下文包
│   ├── compiler.py           # 知识编译与配置指纹
│   ├── api.py                # 本地页面与读写接口
│   ├── cli.py                # analyze/qa/context/timeline 等命令
│   ├── database.py           # schema version 4 与兼容迁移
│   └── static/
│       ├── index.html
│       ├── app.css
│       └── app.js
├── schemas/qa-unit.v0.2.1.schema.json
├── contracts/openapi-v0.3.yaml
├── tests/test_v021_v03.py
├── docs/V0.2.1_CHANGELOG_AND_LEARNING.md
├── docs/V0.3_DEVELOPER_GUIDE.md
└── output/context/学习方法.md
```

13. 自动测试与手动验收

15 项自动测试全部通过，其中 9 项来自 V0.1/V0.2 回归，6 项覆盖新版本。新测试验证 Q/A 单元与 8 条证据关系、第二次分析命中缓存、精确哈希边界、相似标签只生成复核、Context Package 证据完整、本地页面载入、Dashboard 数量、审核写入和时间线坐标。

手动联调使用独立 SQLite 数据库和“学习”示例：导入成功；分析得到 2 个 Q/A 和 6 条主题关系；编译得到 5 个主题、16 条主题证据、5 份文件；第二次分析命中 2 个对话缓存；“学习方法”上下文包包含 2 个 Q/A 与 6 条原句。

页面在真实浏览器中完成视觉检查。桌面宽度下三栏对齐，主题选中状态、总结框、审核状态与证据列表清晰；Q/A 和时间线位于下方。页面同时通过中文内容与 URL 路径联动检查。

14. 隐私、权限与发布边界

服务默认监听 127.0.0.1，页面与 API 同源，V0.3 移除了允许任意网页跨域读取本地数据的响应头。刷新按钮只使用离线规则。系统不获取 Cookie、不保存第三方账号密码、不绕过平台权限、不自动读取用户未导出的聊天。

真实对话的正确入口仍是：用户主动导出文件；官方允许的账号或组织 API；未来明确授权的连接器。浏览器扩展只能作为较晚版本的辅助入口，而且必须限制为用户主动触发、当前页面、明确预览，不得读取登录凭据。

当前可以公开 GitHub 代码，但不应把本地数据库、真实导出、API Key 或个人截图提交。当前可以让少数可信用户在本地试用，但不能作为无鉴权公网 SaaS。

15. 你现在需要人工做什么

第一，运行示例并点击“学习方法”，确认总结框的结构是否符合你的学习习惯。第二，阅读 output/context/学习方法.md，判断它是否足以让另一个 AI 接着工作。第三，用 5-10 条非敏感真实对话做小范围导入，记录错误拆分、漏标签和错误归并。第四，建立 20-30 条人工期望样本，作为下一个版本的评测集。

你暂时不需要学习训练模型，也不需要先搭建 PostgreSQL。最值得先学的是 SQLite 基础查询、JSON、HTTP API、哈希与缓存、多对多关系、最小前端以及如何评价摘要是否忠实。

学习顺序	目标	小练习
1. 数据库关系	看懂 Q/A、主题、证据	画出 qa_units 到 sentence 的路径
2. 哈希与缓存	理解为何避免重复处理	连续运行两次 analyze 比较 cache_hits
3. HTTP 与 JSON	看懂页面如何取数据	浏览器查看 /v0/dashboard
4. 前端基础	能修改总结框布局	改一个标签文字与颜色
5. 评测	判断模型是否真的进步	人工标注 20-30 个样本

16. 版本成熟度判断



V0.2.1 属于可信数据底座，V0.3 属于本地可体验产品 Alpha。相比之前“只有 CLI 的工程 Alpha”，它已经能够在 GitHub 上用页面演示核心差异：跨对话主题、Q/A 拆分、证据回溯和上下文包。

维度	当前判断	说明
产品差异化	8 / 10	不是普通聊天搜索，而是可追溯知识编译
数据设计	8.5 / 10	原文、Q/A、主题、证据、修订、审核分层
本地可用性	7 / 10	有界面和一条命令启动，仍缺安装器
可信度	6.5 / 10	有证据与人工审核，仍缺大规模评测与冲突检测
多平台	4 / 10	架构中立，真实适配器仍少
安全与多用户	3.5 / 10	本地边界清楚，没有登录、租户和云安全
综合阶段	本地 Alpha	可演示、可小范围试用，不可直接商用公网部署

GitHub 是否受欢迎取决于演示和真实解决效果，无法承诺 Star 数量。现在的项目比纯想法或脚手架明显更完整；如果增加双语 README、安装脚本、演示 GIF、真实导入预览和可信评测，它具备获得第一批开源用户的条件。

17. 下一步路线与完整发布预估

建议下一版本不要急着增加更多炫技页面，先完成导入预览、删除同步和评测。之后再加入冲突检测、真实多平台连接器和更强语义归并。

阶段	目标	以每天 2-3 小时估算
V0.3 当前	本地总结、Q/A、时间线、上下文包	已完成
V0.3.1	导入预览、删除同步、失败恢复	1-2 周
V0.4	评测集、冲突检测、语义检索	2-4 周
V0.5	第二/第三个平台真实导入与安装体验	3-5 周
公共 GitHub Beta	文档、演示、隐私检查、Issue 模板	累计约 6-10 周
稳定本地 1.0	性能、备份、迁移、可观测性	累计约 3-4 个月
云 SaaS	登录、多租户、加密、计费、合规	累计约 5-8 个月

18. 本轮最终边界

本轮已经完成：Q/A 单元；问题与回答证据；按对话缓存；标签规范化和哈希；保守的标签合并复核；编译配置指纹；本地 Dashboard；三栏总结框；审核动作；时间线；Context Package；V0.3 API 契约；数据库兼容迁移；15 项测试；浏览器视觉验收；配套开发文档。

本轮没有假装完成：训练模型、自动登录各 AI 平台、全量真实数据连接器、向量语义检索、冲突检测、删除同步、多用户鉴权、云部署、收费系统。把这些明确留在后续版本，是为了让核心“每句话有来源、每份总结可核对”先站稳。

你的产品方向是成立的。真正有价值的不是把聊天搬到另一个列表，而是把问题、答案、主题、时间与来源组织成一套能够持续演化的个人知识系统。V0.2.1 和 V0.3 已经第一次把这个差异完整表现出来。

19. 下一轮人工验收记录模板

第一次使用真实数据时，建议只选择一个低敏感项目并记录以下结果。不要只写“效果不错”，要留下可重复检查的数字和例子。

检查项	记录内容	合格标准
导入完整性	原始对话数、消息数、失败数	数量可与导出文件核对
Q/A 拆分	正确、漏拆、错拆各多少	每个问题能回到原消息
主题标签	正确、过宽、过窄、漏标	重要主题不依赖聊天标题
标签归并	应合并、应分开、系统建议	没有未经人确认的错误合并
摘要忠实度	正确结论、遗漏、捏造	所有核心观点都有原句支撑
Context Package	另一个 AI 是否能继续任务	来源清楚，扩展与原文分开
隐私检查	是否含姓名、账号、密钥、私密信息	演示数据完成脱敏

建议为每个错误保存三项：原始输入、程序输出、你期望的正确结果。累积到 20-30 个样本后，它们就是第一份真正有用的评测集。以后无论更换提示词、模型还是归并算法，都必须在同一评测集上比较，避免“看起来更聪明，实际上把证据弄丢了”。

下一轮开始前，可以先回答四个产品问题：用户最常从哪个平台导入；他们更在乎“找回问过什么”还是“形成新知识”；他们愿意怎样确认标签合并；Context Package 最常被交给哪个 AI 或工作流。这四个答案会决定 V0.3.1 的导入预览和 V0.4 的评测重点。